



DATA STRUCTURES AND ALGORITHMS

23CSE203

Submitted by	
Name	A. Chethan venkata deva
Roll No	AV.SC.U4CSE25008
Year/sem/section	2 nd year/sem-3/CSE-A
Credits	5

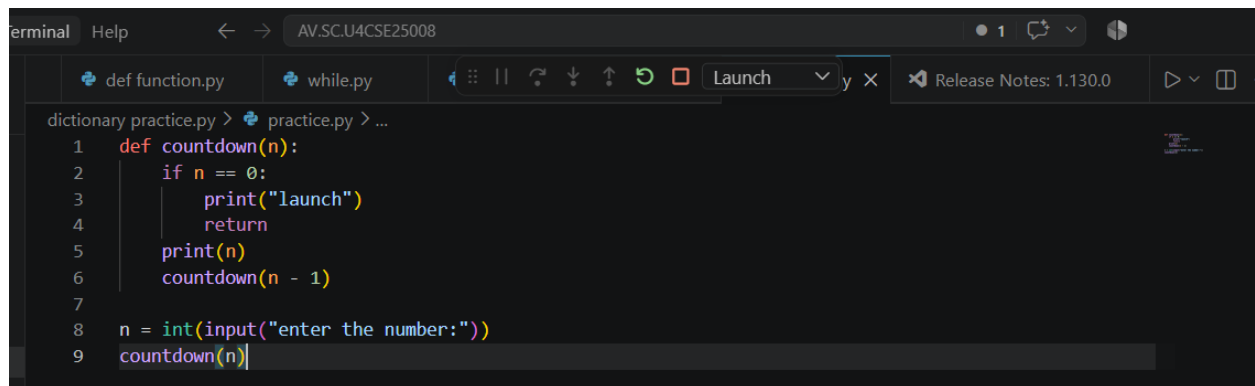
Submitted to	
Name	DR.RAJKUMAR BATCHU
Department	School of computing
Designation	Asst. professor(SI. Grade)

s.no	title	Pageno	remarks
1	WEEK 1: 1a. Recursive countdown from n to Launch. 1b. Recursive calculation of compound growth factor. 1c. Recursive search for employee ID in list. 1d. Recursive factorial calculation for parcel arrangements. 1e. Recursive Fibonacci series generation.		
2	WEEK 2: 2a. Write a python program to perform linear search on the give input. 2b. Write a python program to perform binary search on the given. Verify by taking sorted array as an input. 2c. Write a python program to perform binary search on the given. Verify by taking unsorted array as an input.		
3	WEEK-3: 3a.write a python program to perform bubblesort on the given input 3b. write a python program to perform insertion_sort on the given input 3c. write a python program to perform selection sort by taking given input 3d.write a python program to perform merge sort by taking the given input . 3e. write a python program to perform quick sort by taking given input		

WEEK-1

1 A.AIM: A Rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1 followed by “launch”.

Code:

A screenshot of a code editor window with a dark theme. The window has a title bar with 'terminal', 'Help', and navigation arrows. Below the title bar is a toolbar with icons for file operations and a 'Launch' button. The main area shows a Python file named 'practice.py' with the following code:

```
1 def countdown(n):  
2     if n == 0:  
3         print("launch")  
4         return  
5     print(n)  
6     countdown(n - 1)  
7  
8 n = int(input("enter the number:"))  
9 countdown(n)
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62889' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\practice.py'
enter the number:2
2
1
launch
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> 
```

Explanation

Condition 1: Positive Input ($n > 0$)

- The function checks whether $n == 0$.
- Since n is positive, the condition is **False**.
- It executes:
 - `print(n)` → prints the current value of n .
 - `countdown(n-1)` → calls the same function again with $n-1$.
- This process continues until n becomes 0.
- When $n == 0$, the condition becomes **True**.
- It prints **launch**.
- The return statement stops that function call.
- Then the recursive calls return one by one.
- Since there is no value returned by the function, it returns **None**.

Condition 2: Zero Input ($n == 0$)

- The function checks if $n == 0$.
- The condition is immediately **True**.
- It prints **launch**.
- The return statement stops the function.
- No recursive call takes place.
- The function returns **None**.

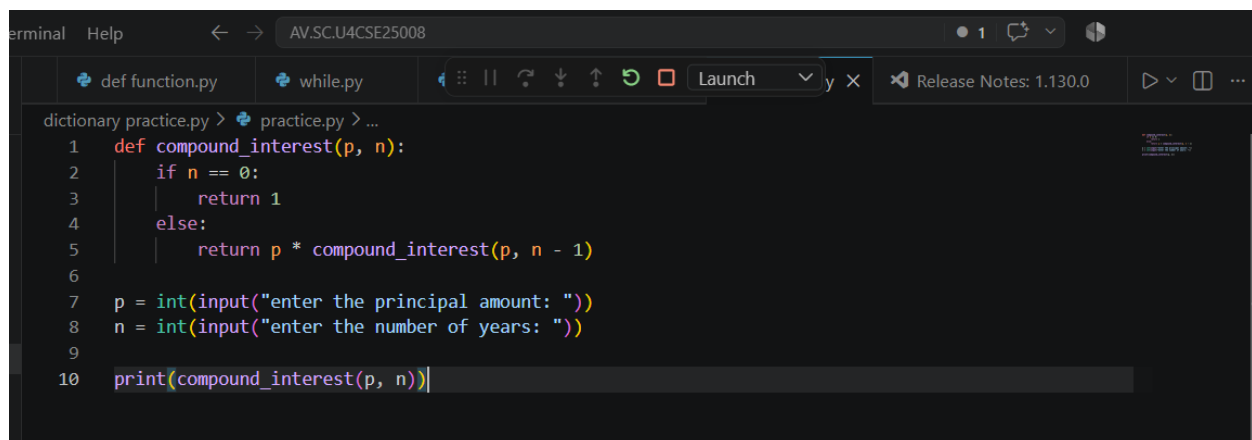
Condition 3: Negative Input ($n < 0$)

- The condition $n == 0$ is **False**.
- It prints the negative value.
- Then it calls itself with $n-1$, making the value even more negative.
- Therefore, it will **never reach 0**.
- The recursion continues indefinitely.
- Eventually, Python raises a **RecursionError** because the maximum recursion depth is exceeded

s.no	Error obtained	Rectification
1	Negative input in countdown	Add a condition to handle negative numbers before recursion.

1B. Aim: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate $p(n)$ where p =principal growth, n = no. of years.

Code:

A screenshot of a code editor window. The title bar shows 'terminal Help' and 'AV.SC.U4CSE25008'. The editor has tabs for 'def function.py', 'while.py', and 'practice.py'. The 'practice.py' tab is active, showing a Python script. The script defines a recursive function 'compound_interest(p, n)' which returns 1 if n is 0, and p * compound_interest(p, n-1) otherwise. Below the function, it takes user input for principal amount and number of years, and prints the result of the recursive function.

```
dictionary practice.py > practice.py > ...
1  def compound_interest(p, n):
2      if n == 0:
3          return 1
4      else:
5          return p * compound_interest(p, n - 1)
6
7  p = int(input("enter the principal amount: "))
8  n = int(input("enter the number of years: "))
9
10 print(compound_interest(p, n))
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'c:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
enter the principal amount: 1000
enter the number of years: 1
1000
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: $n > 0$

- The function `compound_interest(p, n)` is called with the principal amount p and number of years n .
- The function checks whether $n == 0$.
- Since n is greater than 0, the condition is **False**.
- It executes the else block.
- It calculates:
 - $p * \text{compound_interest}(p, n-1)$
- The function calls itself recursively with $n-1$.
- This process continues until n becomes 0.
- Each recursive call multiplies p one more time.
- Therefore, the function calculates p^n .

Condition 2: $n == 0$

- When n becomes 0, the condition $n == 0$ becomes **True**.
- The function returns 1.
- This 1 acts as the **base case** for the multiplication.
- The recursive calls then return one by one.
- Each returned value is multiplied by p .
- Finally, the result is printed using `print(compound_interest(p,n))`

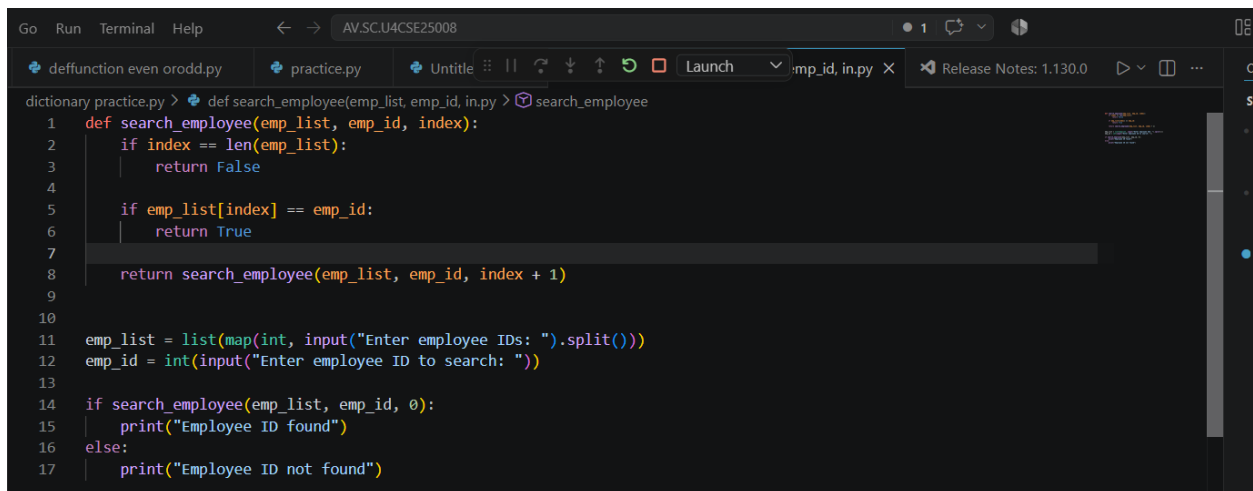
Error table:

s.no	Error obtained	rectification
------	----------------	---------------

1	Wrong indentation	Maintain proper indentation.
---	-------------------	------------------------------

1C. AIM . An Hr department stores employee IDs in a list .Develop a recursive program to search for a given employee ID write a recursive function to search for an employee ID in a list.

Code:

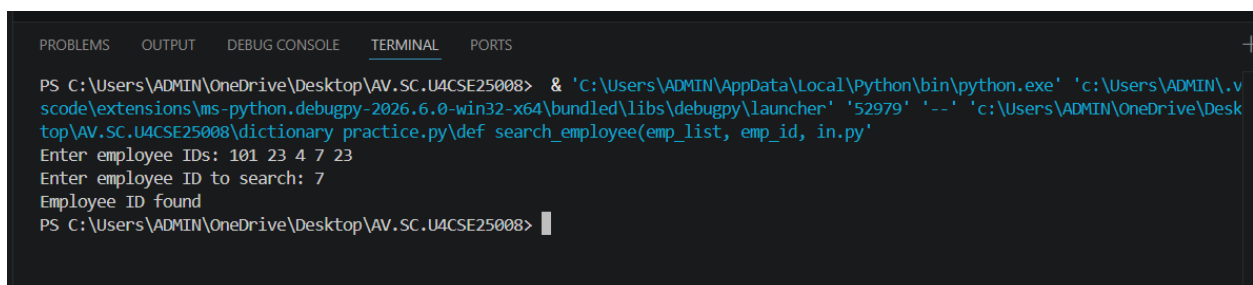


```

Go Run Terminal Help
AV.SC.U4CSE25008
dictionary practice.py > def search_employee(emp_list, emp_id, index):
1 def search_employee(emp_list, emp_id, index):
2     if index == len(emp_list):
3         return False
4
5     if emp_list[index] == emp_id:
6         return True
7
8     return search_employee(emp_list, emp_id, index + 1)
9
10
11 emp_list = list(map(int, input("Enter employee IDs: ").split()))
12 emp_id = int(input("Enter employee ID to search: "))
13
14 if search_employee(emp_list, emp_id, 0):
15     print("Employee ID found")
16 else:
17     print("Employee ID not found")

```

Output:



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'c:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\def search_employee(emp_list, emp_id, in.py'
Enter employee IDs: 101 23 4 7 23
Enter employee ID to search: 7
Employee ID found
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>

```

Explanation

Condition 1: index == len(emp_list)

- The function checks whether index has reached the length of the employee list.
- If index == len(emp_list), it means we have checked **all employee IDs**.
- The employee ID was not found.
- The function returns **False**.

- No further recursion takes place.

Condition 2: `emp_list[index] == emp_id`

- The function checks whether the current employee ID is equal to the ID we are searching for.
- If both IDs are equal, the condition becomes **True**.
- The function returns **True**.
- The recursion stops because the employee ID has been found.

Condition 3: Employee ID does not match

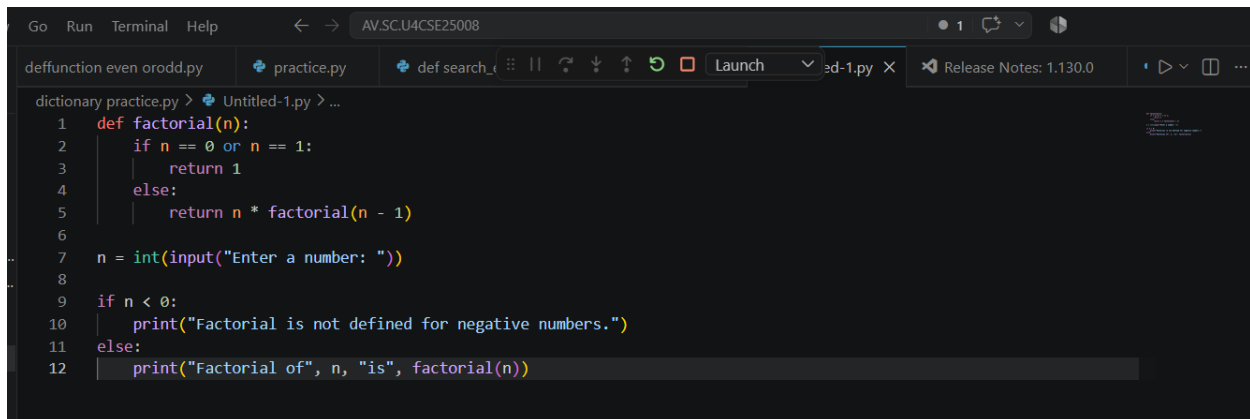
- If the current employee ID is not equal to `emp_id`, the condition is **False**.
- The function calls itself recursively with:
 - `index + 1`
- This moves to the **next employee ID** in the list.
- The process continues until:
 - The employee ID is found, or
 - The entire list is checked.

Error table:

s.no	Error obtained	Rectification
1	Missing return statement	Add the required return statement

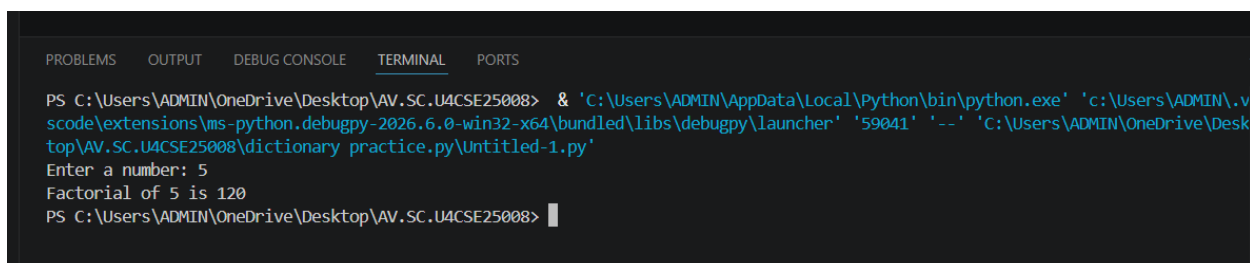
1D. AIM. In a logistics company , the no . of possible ways to arrange in different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

Code:

A screenshot of a code editor window. The top bar shows 'Go Run Terminal Help' and a file path 'AV.SC.U4CSE25008'. Below the top bar, there are tabs for 'dictionary practice.py', 'Untitled-1.py', and 'ed-1.py'. The main editor area shows a Python script for calculating factorials. The code is as follows:

```
1 def factorial(n):
2     if n == 0 or n == 1:
3         return 1
4     else:
5         return n * factorial(n - 1)
6
7 n = int(input("Enter a number: "))
8
9 if n < 0:
10     print("Factorial is not defined for negative numbers.")
11 else:
12     print("Factorial of", n, "is", factorial(n))
```

Output:

A screenshot of a terminal window. The top bar shows 'PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS'. The terminal content is as follows:

```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
Enter a number: 5
Factorial of 5 is 120
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: Negative Input ($n < 0$)

- The program first checks whether $n < 0$.
- If n is negative, the condition is **True**.
- Factorial is not defined for negative numbers.
- It prints:
Factorial is not defined for negative numbers.
- The `factorial()` function is **not called**.

Condition 2: Base Case ($n == 0$ or $n == 1$)

- Inside the `factorial()` function, it checks:
 $n == 0$ or $n == 1$
- If n is 0 or 1, the condition is **True**.
- The function returns **1**.
- This is the **base case** of the recursion.
- It stops further recursive calls.

Condition 3: $n > 1$

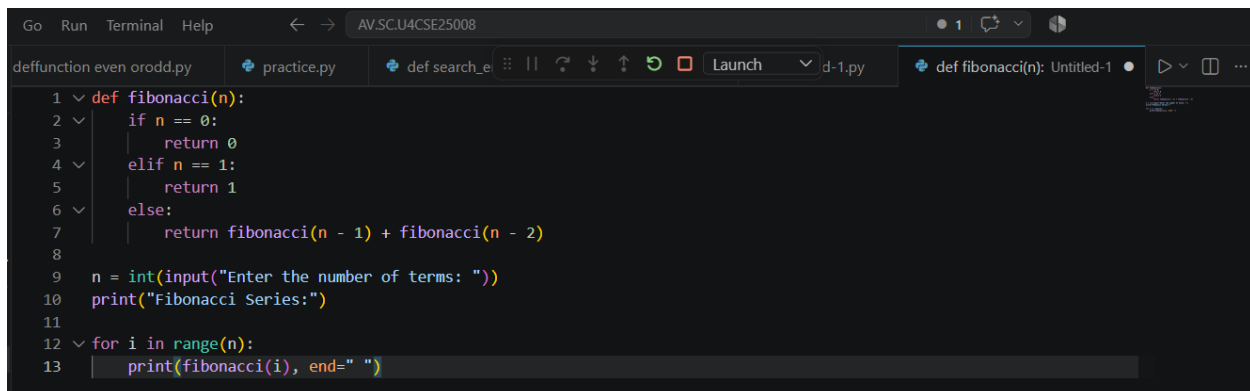
- If n is greater than 1, the condition $n == 0$ or $n == 1$ is **False**.
- The else block is executed.
- It returns:
 $n * \text{factorial}(n - 1)$
- The function calls itself with $n - 1$.
- This continues until n becomes 1.
- Then the returned values are multiplied together

Error table:

s.no	Error obtained	Rectification
1	Wrong recursive value	Updated the recursive value correctly

1E. AIM. Write a recursive program to print the first n terms of the fibonacci series

Code:

A screenshot of a code editor window with a dark theme. The editor shows a Python script for calculating the Fibonacci series. The code is as follows:

```
1 def fibonacci(n):
2     if n == 0:
3         return 0
4     elif n == 1:
5         return 1
6     else:
7         return fibonacci(n - 1) + fibonacci(n - 2)
8
9 n = int(input("Enter the number of terms: "))
10 print("Fibonacci Series:")
11
12 for i in range(n):
13     print(fibonacci(i), end=" ")
```

The editor has multiple tabs open at the top: 'deffunction even or odd.py', 'practice.py', 'def search_e...', and 'def fibonacci(n): Untitled-1'. The 'def fibonacci(n): Untitled-1' tab is currently active. The code is written in a syntax-highlighted style with line numbers on the left.

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '60880' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
Enter the number of terms: 6
Fibonacci Series:
0 1 1 2 3 5
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53127' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
Enter the number of terms: -8
Fibonacci Series:
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: $n == 0$

- The function checks whether $n == 0$.
- If the condition is **True**, the function returns 0.
- This is the **first base case**.
- No further recursive call takes place.

Condition 2: $n == 1$

- If n is not 0, the function checks whether $n == 1$.
- If the condition is **True**, the function returns 1.
- This is the **second base case**.
- No further recursive call takes place

Condition 3: $n > 1$

- If n is neither 0 nor 1, both conditions are **False**.
- The else block is executed.
- The function calls itself **two times**:
 - `fibonacci(n - 1)`
 - `fibonacci(n - 2)`
- The two returned values are added together.

- This process continues until the base cases $n == 0$ and $n == 1$ are reached.

Error table:

s.no	Error obtained	Rectification
1	Incorrect Fibonacci recursion	Use fibonacci(n-1) + fibonacci(n-2).

Week-2

2a. AIM: Write a python program to perform linear search on the give input.

Code:

```
Go Run Terminal Help AV.SC.U4CSE25008
on even or odd.py practice.py def search_em Launch
1 def linear_search(arr, k):
2     for i in range(len(arr)):
3         if arr[i] == k:
4             return i
5     return -1
6
7 n = int(input("enter the no of elements"))
8 arr = []
9
10 for i in range(n):
11     arr.append(int(input("enter the element:")))
12
13 k = int(input("enter the search element"))
14 print(linear_search(arr, k))
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'c:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56540' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
enter the no of elements 2
enter the element:3
enter the element:4
enter the search element 1
-1
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: Element is Found ($\text{arr}[i] == k$)

- The function `linear_search(arr, k)` receives the array `arr` and the search element `k`.
- The for loop starts from the first element of the array.
- It checks each element one by one using:
 $\text{arr}[i] == k$
- If the current element is equal to `k`, the condition becomes **True**.
- The function immediately returns the **index i**.
- Since the function returns the index, the loop stops immediately.
- The remaining elements are not checked.

Condition 2: Element is Not Found

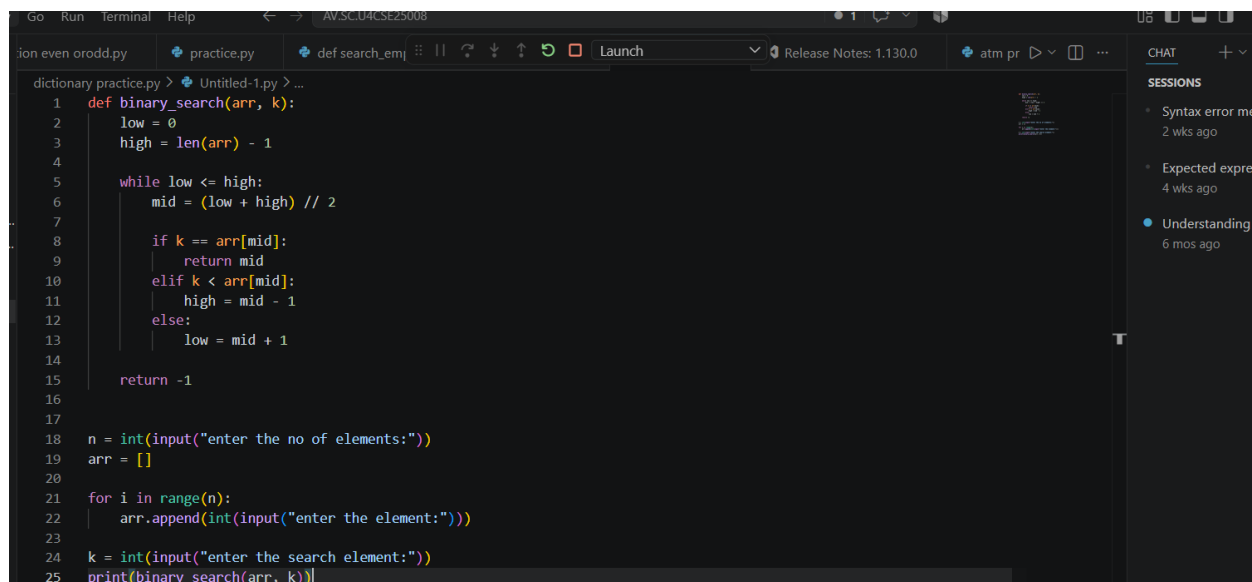
- The loop checks every element in the array.
- If none of the elements is equal to `k`, the if condition is always **False**.
- After the loop finishes, the statement `return -1` is executed.
- `-1` indicates that the search element is **not present** in the array.

Error table:

s.no	Error obtained	Rectification
1	Wrong loop range	Use range(n) to process exactly n elements.

2b. AIM: Write a python program to perform binary search on the given. Verify by taking sorted array as an input.

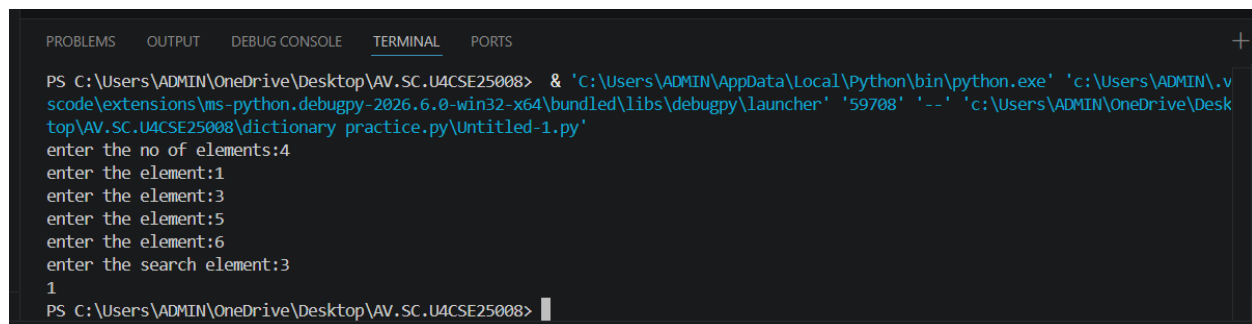
Code:

A screenshot of a code editor window. The editor has a dark theme. The code is written in Python and implements a binary search function. The code is as follows:

```
1 def binary_search(arr, k):
2     low = 0
3     high = len(arr) - 1
4
5     while low <= high:
6         mid = (low + high) // 2
7
8         if k == arr[mid]:
9             return mid
10        elif k < arr[mid]:
11            high = mid - 1
12        else:
13            low = mid + 1
14
15    return -1
16
17
18 n = int(input("enter the no of elements:"))
19 arr = []
20
21 for i in range(n):
22     arr.append(int(input("enter the element:")))
23
24 k = int(input("enter the search element:"))
25 print(binary_search(arr, k))
```

The editor also shows a sidebar on the right with a 'CHAT' section and a 'SESSIONS' list.

Output:

A screenshot of a terminal window. The terminal shows the command prompt and the execution of the Python program. The output is as follows:

```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
enter the no of elements:4
enter the element:1
enter the element:3
enter the element:5
enter the element:6
enter the search element:3
1
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: $k == arr[mid]$ — Element Found

- The function `binary_search(arr, k)` receives the array and the search element.

- low is initialized to 0, which represents the first index.
- high is initialized to len(arr)-1, which represents the last index.
- The while loop runs as long as low <= high.
- mid is calculated using:
mid = (low + high) // 2
- The middle element is compared with the search element k.
- If k == arr[mid], the element is found.
- The function returns the **index mid** and stops searching.

Condition 2: k < arr[mid] — Search Left Half

- If k is smaller than the middle element, the condition is **True**.
- This means the search element can only be present in the **left half**.
- Therefore:
high = mid - 1
- The right half is eliminated.
- The process continues with the remaining left half

Condition 3: k > arr[mid] — Search Right Half

- If k is greater than the middle element, the elif condition is **False**.
- The else block is executed.
- This means the search element can only be present in the **right half**.
- Therefore:
low = mid + 1
- The left half is eliminated.
- The process continues with the remaining right half.

Condition 4: low > high — Element Not Found

- The while loop continues until low <= high becomes **False**.
- When low > high, there are no elements left to search.
- This means the search element is not present in the array.
- The function returns **-1**.

Error table:

S.no	Error obtained	Rectification
	Wrong index in binary search	Use high = mid - 1 for the left half and low = mid + 1 for the right half.

```

1 def binary_search(arr, key):
2     low = 0
3     high = len(arr) - 1
4
5     while low <= high:
6         mid = (low + high) // 2
7
8         if arr[mid] == key:
9             return mid
10        elif key < arr[mid]:
11            high = mid - 1
12        else:
13            low = mid + 1
14
15    return -1
16
17
18 arr = list(map(int, input("Enter the elements: ").split()))
19
20 if arr == sorted(arr):
21     print("Array is already sorted.")
22 else:
23     print("Array is unsorted.")
24     arr.sort()
25     print("Array after sorting:", arr)
26
27 key = int(input("Enter the element to search: "))
28
29 result = binary_search(arr, key)
30
31 if result != -1:
32     print("Element found at index:", result)
33 else:
34     print("Element not found.")

```

Output:

```

PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> ^C
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> c:: cd 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008'; & 'c:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55895' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
Enter the elements: 1 4 2 6 8
Array is unsorted.
Array after sorting: [1, 2, 4, 6, 8]
Enter the element to search: 2
Element found at index: 1
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>

```

Explanation

Condition 1: $\text{arr}[\text{mid}] == \text{key}$ — Element Found

- The function first calculates the middle index using $\text{mid} = (\text{low} + \text{high}) // 2$.
- It compares the middle element $\text{arr}[\text{mid}]$ with the search element key .
- If $\text{arr}[\text{mid}] == \text{key}$, the condition is **True**.

- The element is found.
- The function returns the index mid.
- The search stops.

Condition 2: $\text{key} < \text{arr}[\text{mid}]$ — Search Left Half

- If key is smaller than $\text{arr}[\text{mid}]$, the condition is **True**.
- This means the required element may be present in the **left half**.
- The right half is ignored.
- high is changed to $\text{mid} - 1$.
- The search continues in the left half.

Condition 3: $\text{key} > \text{arr}[\text{mid}]$ — Search Right Half

- If key is greater than $\text{arr}[\text{mid}]$, the previous conditions are **False**.
- The element may be present in the **right half**.
- The left half is ignored.
- low is changed to $\text{mid} + 1$.
- The search continues in the right half.

Condition 4: $\text{low} > \text{high}$ — Element Not Found

- The while $\text{low} \leq \text{high}$ loop continues until there are elements left to search.
- When $\text{low} > \text{high}$, the loop stops.
- This means the element is not present in the array.
- The function returns -1.
- The program prints "**Element not found.**"

Condition 5: Array is Already Sorted

- The program checks whether the given array is already sorted.
- It uses:
`arr == sorted(arr)`
- If the condition is **True**, it prints:
"**Array is already sorted.**"
- The array is directly used for binary search.

Condition 6: Array is Unsorted

- If `arr == sorted(arr)` is **False**, the array is unsorted.
- The program prints:
"Array is unsorted."
- It then uses `arr.sort()` to sort the array.
- The sorted array is displayed.
- Binary search is then performed on the sorted array.

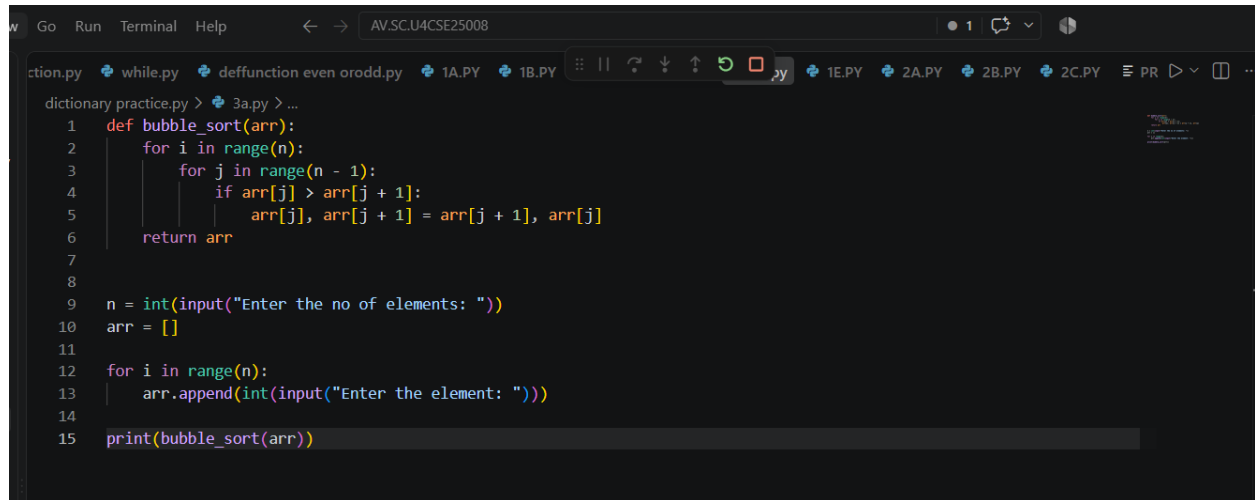
Error table:

s.no	Error obtained	Rectification
1	Binary search on unsorted array	sort the array first using <code>arr.sort()</code> or provide a sorted array.

Week-3

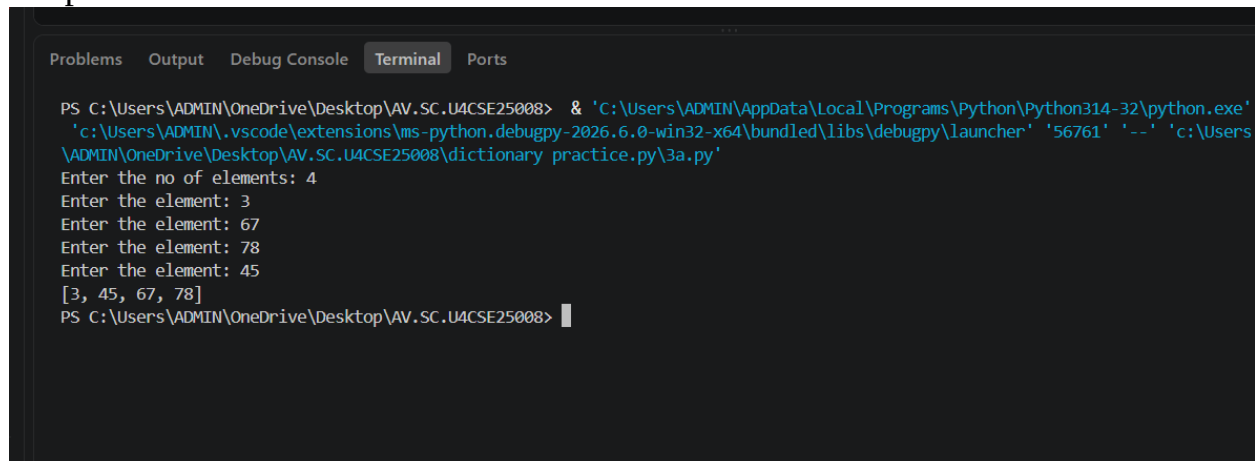
3a.Aim.Write a python program to sort the elements using a bubble sort technique. Take an input elements from the user.

Code:

A screenshot of a Visual Studio Code editor window. The file explorer on the left shows a project named 'dictionary practice.py' with a sub-file '3a.py'. The main editor area displays the code for '3a.py'. The code defines a 'bubble_sort' function that takes an array 'arr' and its length 'n' as input. It uses two nested loops: an outer loop for 'i' from 0 to 'n-1' and an inner loop for 'j' from 0 to 'n-i-1'. Inside the inner loop, it checks if 'arr[j] > arr[j+1]'. If true, it swaps the elements at indices 'j' and 'j+1'. After the loops, it returns the sorted array. The main part of the code prompts the user to enter the number of elements, then enters a loop to collect each element into an array 'arr'. Finally, it prints the result of 'bubble_sort(arr)'.

```
1 def bubble_sort(arr):
2     for i in range(n):
3         for j in range(n - 1):
4             if arr[j] > arr[j + 1]:
5                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
6     return arr
7
8
9 n = int(input("Enter the no of elements: "))
10 arr = []
11
12 for i in range(n):
13     arr.append(int(input("Enter the element: ")))
14
15 print(bubble_sort(arr))
```

Output:

A screenshot of a terminal window showing the execution of the Python program. The prompt is 'PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>'. The user enters '4' for the number of elements. Then, they enter '3', '67', '78', and '45' as the elements. The program outputs the sorted array '[3, 45, 67, 78]'.

```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'c:\Users\ADMIN\AppData\Local\Programs\Python\Python314-32\python.exe'
'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56761' '--' 'c:\Users
\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\3a.py'
Enter the no of elements: 4
Enter the element: 3
Enter the element: 67
Enter the element: 78
Enter the element: 45
[3, 45, 67, 78]
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: $arr[j] > arr[j+1]$ — Elements Need To Be Swapped

- The function `bubble_sort(arr)` receives the array as an argument.
- The outer for loop runs from 0 to $n-1$ and controls the number of passes.
- The inner for loop runs from 0 to $n-2$ and compares adjacent elements.
- The condition $arr[j] > arr[j+1]$ checks whether the current element is greater than the next element.
- If the condition is **True**, the two elements are in the wrong order.
- Therefore, the two elements are swapped using:

- `arr[j],arr[j+1]=arr[j+1],arr[j]`
- After the swap, the smaller element moves toward the left and the larger element moves toward the right.

Condition 2: `arr[j] <= arr[j+1]` — No Swap Required

- If `arr[j] > arr[j+1]` is **False**, it means the current element is smaller than or equal to the next element.
- Therefore, the elements are already in the correct ascending order.
- No swapping is performed.
- The inner loop continues to compare the next pair of adjacent elements.

Condition 3: `for i in range(n)` — Repeated Passes

- The outer loop repeats the comparison process for all the elements.
- After each complete pass, the **largest unsorted element moves to the end of the array**.
- The process is repeated until all elements are arranged in ascending order.
- Finally, the function returns the sorted array using:

Error table:

sno	Error obtained	Rectification
1	IndexError: list index out of range	Use correct loop range

3b.Aim. write a python program to sort the elements using a insertion sort technique. Take an input elements from the user.

Code:

```

ew  Go  Run  Terminal  Help  <  >  AV.SC.U4CSE25008  ● 1  ↺  ↻  🗑️
def insertion_sort(arr):
    for i in range(1, n):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1

        arr[j + 1] = key

    return arr

n = int(input("Enter the number of elements: "))
arr = []

for i in range(n):
    arr.append(int(input("Enter the element: ")))

print(insertion_sort(arr))

```

Output:

```
21 print(insertion_sort(arr))

Problems Output Debug Console Terminal Ports

PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Programs\Python\Python314-32\python.exe'
'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62633' '--' 'c:\Users
\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary_practice.py\3b.py'
Enter the number of elements: 5
Enter the element: 23
Enter the element: 45
Enter the element: 67
Enter the element: 12
Enter the element: 45
[12, 23, 45, 45, 67]
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> |
```

Explanation:

Condition 1: for i in range(1, len(arr)) — Selecting Element

- The loop starts from the **second element**.
- The first element is considered already sorted.
- i represents the element that needs to be placed correctly.

Condition 2: key = arr[i] — Storing Element

- The current element is stored in key.

key = arr[i]

- This element will be compared with the elements before it.

Condition 3: j = i - 1 — Getting Previous Position

- j points to the element just before the current element.

j = i - 1

Condition 4: while j >= 0 and key < arr[j] — Comparing Elements

- The current element is compared with the previous elements.
- If the previous element is greater than key, it is moved one position to the right.

while j >= 0 and key < arr[j]:

arr[j + 1] = arr[j]

j -= 1

- This creates space for the current element.

Condition 5: arr[j + 1] = key — Inserting Element

- The key is placed in its correct position.

arr[j + 1] = key

- The elements before the current position are now sorted.

Condition 6: return arr — Returning Sorted Array

- After all elements are processed, the function returns the sorted array.

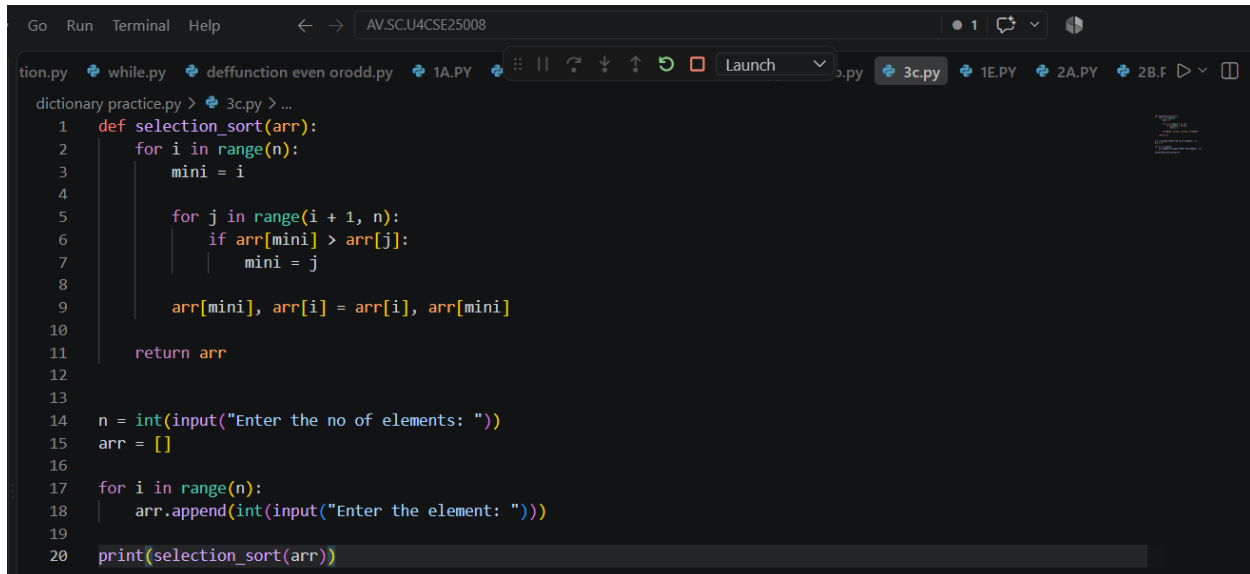
return arr

error table:

sno	Error obtained	Rectification
1	Elements not sorted correctly	Shift elements before insertion

3c.write a python program to sort the elements using selection sort technique . take an input from the user.

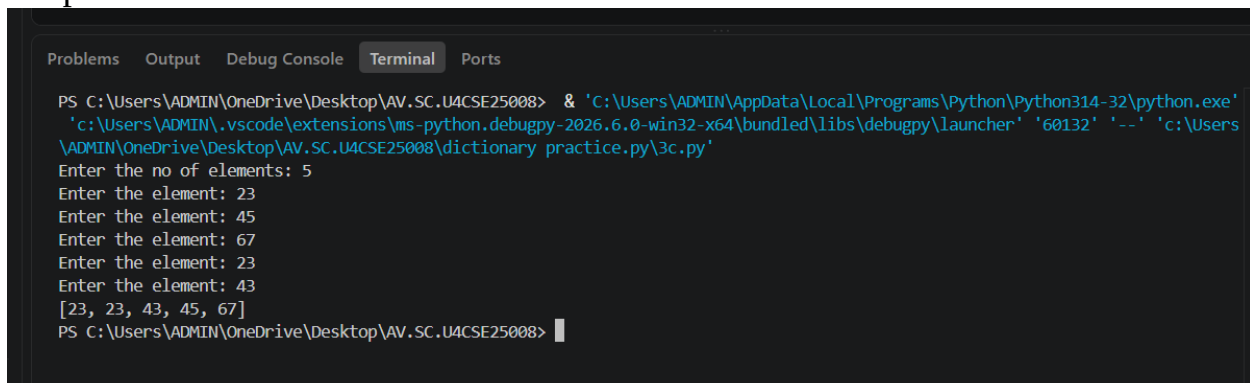
Code:

A screenshot of a Python IDE with a dark theme. The editor shows a file named '3c.py' containing a selection sort algorithm. The code is as follows:

```
1 def selection_sort(arr):
2     for i in range(n):
3         mini = i
4
5         for j in range(i + 1, n):
6             if arr[mini] > arr[j]:
7                 mini = j
8
9         arr[mini], arr[i] = arr[i], arr[mini]
10
11     return arr
12
13
14 n = int(input("Enter the no of elements: "))
15 arr = []
16
17 for i in range(n):
18     arr.append(int(input("Enter the element: ")))
19
20 print(selection_sort(arr))
```

The IDE interface includes a menu bar (Go, Run, Terminal, Help), a toolbar with icons for file operations and execution, and a sidebar showing the file explorer.

Output:

A screenshot of a terminal window showing the execution of the Python program. The prompt is 'PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>'. The user enters the command to run the program. The output shows the program asking for the number of elements (5) and then five elements (23, 45, 67, 23, 43). The final output is the sorted array: [23, 23, 43, 45, 67].

```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Programs\Python\Python314-32\python.exe'
'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '60132' '--' 'c:\Users
\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\3c.py'
Enter the no of elements: 5
Enter the element: 23
Enter the element: 45
Enter the element: 67
Enter the element: 23
Enter the element: 43
[23, 23, 43, 45, 67]
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation:

Condition 1: for i in range(n) — Selecting Position

- The loop goes through each position in the array.
- i represents the current position where the smallest element will be placed.

Condition 2: mini = i — Assuming Minimum

- Initially, the element at index i is assumed to be the smallest.

Condition 3: for j in range(i+1,n) — Finding Minimum

- The loop checks all elements after position i.
- It searches for the smallest element.

Condition 4: if arr[mini] > arr[j] — Comparing

- Compares the current minimum with arr[j].
- If arr[j] is smaller, mini is updated.

Condition 5: mini = j — Updating Minimum

- Stores the index of the newly found smallest element.

Condition 6: arr[mini], arr[i] = arr[i], arr[mini] — Swapping

- The smallest element is swapped with the element at position i.
- This places the smallest element in its correct position.

Condition 7: return arr — Returning Result

- Returns the completely sorted array.

Error table:

Sno	Error obtained	Rectification
1	Wrong minimum value	Update min_index properly

3d.write a python program to sort the elements using the merge sort technique . take an input from the user.

Code:

```
ew Go Run Terminal Help AV.SC.U4CSE25008
while.py deffunction even orodd.py 1A.PY 1B.PY 3c.py 3d.py 1E.PY 2A.PY 2B.PY 2C.PY PRINT("HELLO"); Un
dictionary practice.py > 3d.py > merge_sort
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = arr[:mid]
6     right = arr[mid:]
7
8     merge_sort(left)
9     merge_sort(right)
10
11     i = j = k = 0
12     while i < len(left) and j < len(right):
13         if left[i] < right[j]:
14             arr[k] = left[i]
15             i += 1
16         else:
17             arr[k] = right[j]
18             j += 1
19             k += 1
20
21     while i < len(left):
22         arr[k] = left[i]
23         i += 1
24         k += 1
25
26     while j < len(right):
27         arr[k] = right[j]
28         j += 1
29         k += 1
30
31 n = int(input("Enter the no of elements: "))
32 arr = []
33 for i in range(n):
34     arr.append(int(input("Enter the element: ")))
35 merge_sort(arr)
36 for element in arr:
37     print(element)
```

Output:

```
Problems Output Debug Console Terminal Ports
Enter the no of elements: 4
Enter the element: 34
Enter the element: 23
Enter the element: 12
Enter the element: 45
12
23
34
45
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation:

Condition 1: if len(arr) <= 1 — Checking the Base Condition

- The function first checks whether the array contains **one or zero elements**.

if len(arr) <= 1:

 return arr

- If the array has one or zero elements, it is already sorted.
- No further division or comparison is required.
- The function returns the array as it is:

return arr

Condition 2: mid = len(arr) // 2 — Finding the Middle

- The array is divided into two parts.
- The middle position is calculated using:

mid = len(arr) // 2

- len(arr) gives the total number of elements.
- // 2 performs integer division and finds the middle index.

Condition 3: left = arr[:mid] — Creating the Left Part

- The elements before the middle position are stored in left.

left = arr[:mid]

Condition 4: right = arr[mid:] — Creating the Right Part

- The elements from the middle position to the end are stored in right.

right = arr[mid:]

Condition 5: merge_sort(left) and merge_sort(right) — Sorting Both Parts

- The left and right parts are recursively sorted.

merge_sort(left)

merge_sort(right)

- The array continues to be divided until each part contains only **one element**.
- A single element is already sorted.
- The smaller sorted parts are then merged together.

Condition 6: i = j = k = 0 — Initializing Indexes

- Three variables are initialized:

i = j = k = 0

- i keeps track of the current element in the **left array**.
- j keeps track of the current element in the **right array**.
- k keeps track of the position in the **original array** where the element will be placed.

Condition 7: while i < len(left) and j < len(right) — Comparing Both Parts

- The program compares elements from the left and right arrays.

while i < len(left) and j < len(right):

- The loop continues as long as both arrays still have elements to compare.
- This allows the program to select the smaller element and place it into the original array.

Condition 8: if left[i] < right[j] — Comparing Elements

- The current elements of the left and right arrays are compared.

if left[i] < right[j]:

- If the left element is smaller than the right element, it is placed into the original array.

arr[k] = left[i]

- Then i is increased:

i += 1

- This moves to the next element of the left array.

Condition 9: else — Taking the Right Element

- If the right element is smaller than or equal to the left element, the right element is placed into the original array.

else:

arr[k] = right[j]

- Then j is increased:

j += 1

- This moves to the next element of the right array.

Condition 10: k += 1 — Moving to the Next Position

- After placing an element into the original array, k is increased.

k += 1

Condition 11: while i < len(left) — Copying Remaining Left Elements

- Sometimes the right array becomes empty before the left array.

while i < len(left):

- Any remaining elements in the left array are copied into the original array.

arr[k] = left[i]

- Then both i and k are increased:

i += 1

k += 1

- The remaining elements are already sorted, so no comparison is needed.

Condition 12: while j < len(right) — Copying Remaining Right Elements

- Sometimes the left array becomes empty before the right array.

while j < len(right):

- Any remaining elements in the right array are copied into the original array.

arr[k] = right[j]

- Then j and k are increased:

j += 1

k += 1

- This completes the merging process.

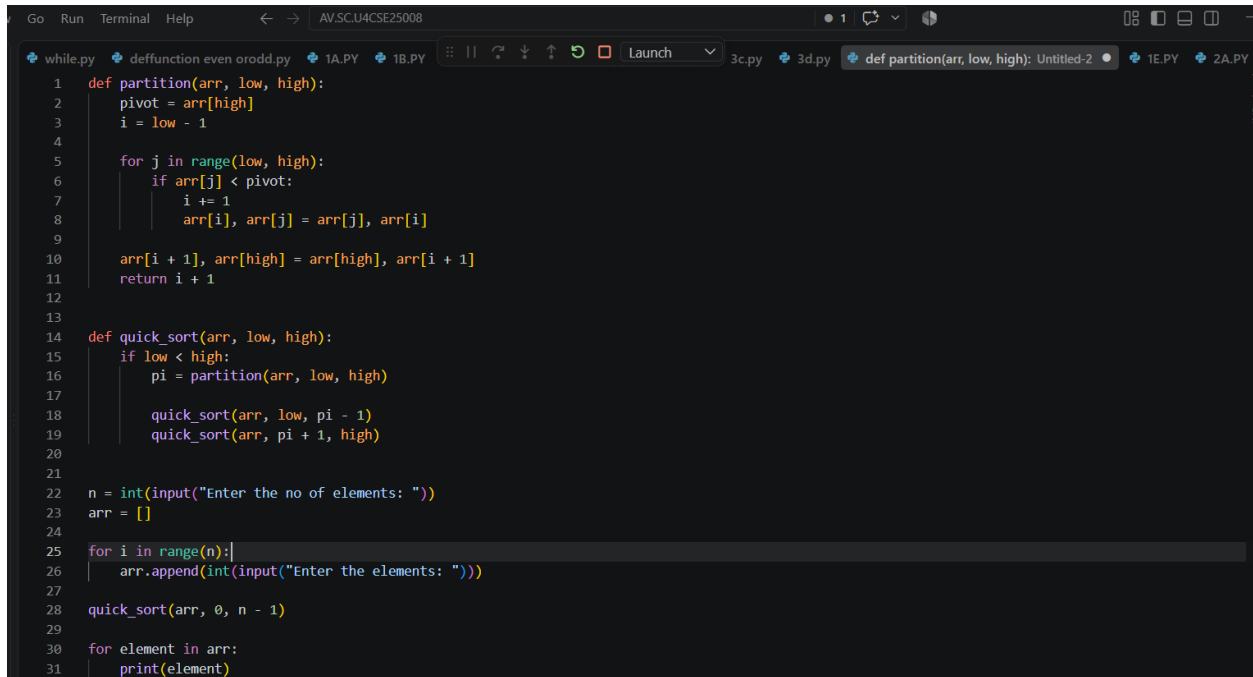
Error table:

sno	Error obtained	Rectification
1	Recursive error	Use if len(arr) <= 1
2	IndexError	Check left/right array indices

Bottom of Form

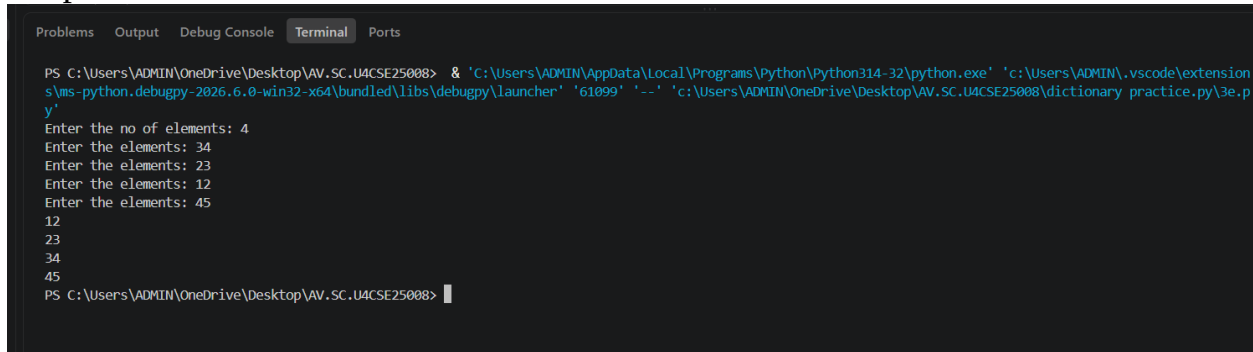
3e. Aim. Write a python program to sort the elements using a quick sort technique . take an input elements from the user

Code:

A screenshot of a Visual Studio Code editor window. The editor is open to a file named 'Untitled-2' which contains a Python program for quick sort. The code is as follows:

```
1 def partition(arr, low, high):
2     pivot = arr[high]
3     i = low - 1
4
5     for j in range(low, high):
6         if arr[j] < pivot:
7             i += 1
8             arr[i], arr[j] = arr[j], arr[i]
9
10    arr[i + 1], arr[high] = arr[high], arr[i + 1]
11    return i + 1
12
13
14 def quick_sort(arr, low, high):
15     if low < high:
16         pi = partition(arr, low, high)
17
18         quick_sort(arr, low, pi - 1)
19         quick_sort(arr, pi + 1, high)
20
21
22 n = int(input("Enter the no of elements: "))
23 arr = []
24
25 for i in range(n):
26     arr.append(int(input("Enter the elements: ")))
27
28 quick_sort(arr, 0, n - 1)
29
30 for element in arr:
31     print(element)
```

Output:

A screenshot of a terminal window showing the execution of the Python program. The prompt is 'PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>'. The user enters '4' for the number of elements. Then, the user enters the elements '34', '23', '12', and '45' one by one. The program outputs the sorted array: '12', '23', '34', '45'. The prompt returns to 'PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>'.

```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Programs\Python\Python314-32\python.exe' 'c:\Users\ADMIN\.vscode\extension
s\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '61899' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\3e.p
y'
Enter the no of elements: 4
Enter the elements: 34
Enter the elements: 23
Enter the elements: 12
Enter the elements: 45
12
23
34
45
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: if arr[j] < pivot — Element Smaller Than Pivot

- The function partition(arr, low, high) is used to divide the array into two parts.
- pivot = arr[high] selects the **last element** of the array as the pivot.
- i = low - 1 is initialized to keep track of the position where elements smaller than the pivot should be placed.
- The for loop checks each element from index low to high-1.
- The condition arr[j] < pivot checks whether the current element is smaller than the pivot.
- If the condition is **True**, i is increased by 1.
- The current element is then swapped with the element at index i:

`arr[i], arr[j] = arr[j], arr[i]`

- This places all elements smaller than the pivot toward the left side.

Condition 2: `arr[j] >= pivot` — No Swap Required

- If `arr[j] < pivot` is **False**, it means the current element is greater than or equal to the pivot.
- No swapping is performed.
- The loop continues to check the next element.
- These elements will remain on the right side of the pivot after partitioning.

Condition 3: `arr[i+1], arr[high] = arr[high], arr[i+1]` — Placing the Pivot

- After all elements have been checked, the pivot is placed in its correct position.
- The pivot is swapped with the element at index `i+1`:

`arr[i+1], arr[high] = arr[high], arr[i+1]`

- After this operation:
 - Elements smaller than the pivot are on the **left**.
 - The pivot is in its **correct position**.
 - Elements greater than or equal to the pivot are on the **right**.
- The function returns the pivot position:

`return i+1`

Condition 4: if `low < high` — Continue Quick Sort

- The function `quick_sort(arr, low, high)` is used to sort the array.
- The condition `low < high` checks whether the current part of the array contains more than one element.
- If the condition is **True**, the array needs to be sorted.
- The `partition()` function is called:

`pi = partition(arr, low, high)`

- `pi` stores the final position of the pivot.

Condition 5: `quick_sort(arr, low, pi-1)` — Sort Left Part

- After partitioning, the elements on the left side of the pivot are sorted separately.
- This is done using:

`quick_sort(arr, low, pi-1)`

- The pivot itself is not included because it is already in its correct position.

Condition 6: `quick_sort(arr, pi+1, high)` — Sort Right Part

- The elements on the right side of the pivot are sorted separately.
- This is done using:

`quick_sort(arr, pi+1, high)`

- Again, the pivot is not included because it is already correctly positioned.

Input Section

- `n = int(input("enter the no of elements:"))` takes the number of elements from the user.
- `arr = []` creates an empty list to store the elements.
- The for loop runs `n` times and takes each element from the user:
- `for i in range(n):`

`arr.append(int(input("enter the elements:")))`

- Each entered value is converted into an integer and added to the array using `append()`.

Calling the Quick Sort Function

`quick_sort(arr, 0, n-1)`

- This calls the `quick_sort()` function.

- 0 represents the first index of the array.
- n-1 represents the last index of the array.
- The complete array is therefore passed for sorting.

Error table:

sno	Error obtained	Rectification
1	IndexError	Check i, j, and pivot boundaries
2	Wrong swapping conditions	By shifting to the correct spaces

WEEK-4

AIM: write a python program to implement a single linked list which could perform the below operations by taking an input from the user

- i. create a linked list
- ii. insert at beginning
- iii. insert at end
- iv. insert at specific index
- v. delete by value
- vi. delete at beginning
- vii. delete at the end
- viii. count no of nodes
- ix. display or traverse
- x. exit

code:

```
sll.py - C:/Users/cse25008/OneDrive/Desktop/dsa lab/week4/sll.py (3.13.7)
File Edit Format Run Options Window Help

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    # i. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            value = int(input("Enter value: "))
            self.insert_end(value)

    # ii. Insert at beginning
    def insert_beginning(self, value):
        new_node = Node(value)
        new_node.next = self.head
        self.head = new_node

    # iii. Insert at end
    def insert_end(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            return

        current = self.head

        while current.next:
            current = current.next

        current.next = new_node
```

```

# iv. Insert at specific index
def insert_at_index(self, value, index):
    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_beginning(value)
        return

    current = self.head

    for _ in range(index - 1):
        if current is None:
            print("Index out of range")
            return
        current = current.next

    if current is None:
        print("Index out of range")
        return

    new_node = Node(value)
    new_node.next = current.next
    current.next = new_node

# v. Delete by value
def delete_by_value(self, value):
    if self.head is None:
        print("List is empty")
        return

    if self.head.data == value:
        self.head = self.head.next
        return

    current = self.head

    while current.next and current.next.data != value:
        current = current.next

    if current.next is None:
        print("Value not found")
    else:
        current.next = current.next.next

```

```
# vi. Delete at beginning
def delete_beginning(self):
    if self.head is None:
        print("List is empty")
        return

    self.head = self.head.next

# vii. Delete at end
def delete_end(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head.next is None:
        self.head = None
        return

    current = self.head

    while current.next.next:
        current = current.next

    current.next = None

# viii. Count number of nodes
def count_nodes(self):
    count = 0
    current = self.head

    while current:
        count += 1
        current = current.next

    return count
```

```

# ix. Display / Traverse
def display(self):
    if self.head is None:
        print("List is empty")
        return

    current = self.head

    while current:
        print(current.data, end=" -> ")
        current = current.next

    print("None")

linked_list = LinkedList()

while True:

    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete at beginning")
    print("7. Delete at end")
    print("8. Count nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        linked_list.create()

    elif choice == 2:
        value = int(input("Enter value: "))
        linked_list.insert_beginning(value)

    elif choice == 3:
        value = int(input("Enter value: "))
        linked_list.insert_end(value)

```

```

linked_list.insert_at_index(value,
elif choice == 4:
    value = int(input("Enter value: "))
    index = int(input("Enter index: "))
    linked_list.insert_at_index(value, index)

elif choice == 5:
    value = int(input("Enter value to delete: "))
    linked_list.delete_by_value(value)

elif choice == 6:
    linked_list.delete_beginning()

elif choice == 7:
    linked_list.delete_end()

elif choice == 8:
    print("Number of nodes:", linked_list.count_nodes())

elif choice == 9:
    linked_list.display()

elif choice == 10:
    print("Program exited.")
    break

else:
    print("Invalid choice")

```

Output:

Creating a linked list

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter value: 1
Enter value: 2
Enter value: 3
```

Inserting at the beginnig

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter value: 6
```

Insertion at the end:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter value: 5
```

Insertion at specific index:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter value: 2
Enter index: 2
```

Delete at beginning:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
```

Delete at the end:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
```

Delete at the specific index:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 5
```

Number of nodes:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 3
```

Display all the nodes:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
1 -> 2 -> 2 -> None
```

Exiting the program:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program exited.
```

Explanation:

1. A **singly linked list** is a collection of nodes where each node contains **two parts**: data and a link to the next node.
2. Each node contains:
 - **Data** – stores the value.
 - **Next (next)** – stores the reference/address of the next node.
3. The program maintains two references:
 - **Head** – points to the first node.
 - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and store the data and reference to the next node.
6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are created one by one and added to the list.
7. For the first node, both **head and tail** point to the same node, and the next of the tail is None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node's next points to the current head, and the new node becomes the new head.
9. During **insertion at the end**, the new node is added after the current tail. The current tail's next points to the new node, and the new node becomes the new tail.
10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the appropriate nodes by adjusting their next links.
11. During **deletion by value**, the list is searched for the node containing the given value. The node is removed by making the previous node point to the node after the deleted node.
12. If the node to be deleted is the **head**, the next node becomes the new head.

13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
16. During **deletion at the end**, the list is traversed to find the node before the tail. That node becomes the new tail and its next is set to None.
17. The **count operation** traverses the list from the head to the end and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves through the list using the next link until None is reached.
20. The program continues to execute until the user selects the **Exit** option.

Error table:

sno	Error obtained	rectification
1	Infinite loop or incomplete traversal due to not updating current	Move to the next node using temp = temp.next
2	Forgetting to update head while inserting/deleting at the beginning	Update head to point to the new first node using head = new_node or head = head.next

AIM: write a python program to implement a double linked list which could perform the below operations by taking an input from the user

i. create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

vii. delete at the end

viii. count no of nodes

ix. display or traverse

x. exit

code:

cse25008.py - C:/Users/cse25008/OneDrive/Desktop/dsa lab/week4/dll.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # i. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for _ in range(n):
            value = int(input("Enter value: "))
            self.insert_end(value)

    # ii. Insert at beginning
    def insert_beginning(self, value):
        new_node = Node(value)

        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node

        self.head = new_node

    # iii. Insert at end
    def insert_end(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            return

        current = self.head

        while current.next:
            current = current.next

        current.next = new_node
        new_node.prev = current
```

```
# iv. Insert at specific index
def insert_at_index(self, value, index):
    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_beginning(value)
        return

    current = self.head

    for _ in range(index - 1):
        if current is None:
            print("Index out of range")
            return
        current = current.next

    if current is None:
        print("Index out of range")
        return

    new_node = Node(value)

    new_node.next = current.next
    new_node.prev = current

    if current.next is not None:
        current.next.prev = new_node

    current.next = new_node
```

```

# v. Delete by value
def delete_by_value(self, value):
    if self.head is None:
        print("List is empty")
        return

    current = self.head

    while current and current.data != value:
        current = current.next

    if current is None:
        print("Value not found")
        return

    if current.prev is not None:
        current.prev.next = current.next
    else:
        self.head = current.next

    if current.next is not None:
        current.next.prev = current.prev

# vi. Delete at beginning
def delete_beginning(self):
    if self.head is None:
        print("List is empty")
        return

    self.head = self.head.next

    if self.head is not None:
        self.head.prev = None

```

```

# vii. Delete at end
def delete_end(self):
    if self.head is None:
        print("List is empty")
        return

    current = self.head

    while current.next:
        current = current.next

    if current.prev is None:
        self.head = None
    else:
        current.prev.next = None

# viii. Count nodes
def count_nodes(self):
    count = 0
    current = self.head

    while current:
        count += 1
        current = current.next

    return count

# ix. Display / Traverse
def display(self):
    if self.head is None:
        print("List is empty")
        return

    current = self.head

    while current:
        print(current.data, end=" <-> ")
        current = current.next

    print("None")

```

```
l11 = DoublyLinkedList()
```

```
print("2. Insert at beginning")
print("3. Insert at end")
print("4. Insert at specific index")
print("5. Delete by value")
print("6. Delete at beginning")
print("7. Delete at end")
print("8. Count nodes")
print("9. Display / Traverse")
print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    dll.create()

elif choice == 2:
    value = int(input("Enter value: "))
    dll.insert_beginning(value)

elif choice == 3:
    value = int(input("Enter value: "))
    dll.insert_end(value)

elif choice == 4:
    value = int(input("Enter value: "))
    index = int(input("Enter index: "))
    dll.insert_at_index(value, index)

elif choice == 5:
    value = int(input("Enter value to delete: "))
    dll.delete_by_value(value)

elif choice == 6:
    dll.delete_beginning()

elif choice == 7:
    dll.delete_end()

elif choice == 8:
    print("Number of nodes:", dll.count_nodes())

elif choice == 9:
    dll.display()

elif choice == 10:
    print("Program exited.")
    break

else:
    print("Invalid choice")
```

Output:

Creating a linked list:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter value: 1
Enter value: 2
Enter value: 3
```

Inserting at the beginning:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter value: 6
```

Insert at the end:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter value: 8
```

Insert at the specific index:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter value: 44
Enter index: 4
```

Delete By value

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 44
```

Delete at the beginning:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
```

Delete at the end:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
```

Number of nodes:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 3
```

Display the nodes:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
1 <-> 2 <-> 3 <-> None
```

Exiting the program:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program exited.
```

Explanation:

1. A **doubly linked list** is a collection of nodes where each node contains **three parts**: data, a link to the previous node, and a link to the next node.
2. Each node has:
 - **Data** – stores the value.
 - **Previous (prev)** – points to the previous node.
 - **Next (next)** – points to the next node.
3. The program maintains two references:
 - **Head** – points to the first node.
 - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and maintain the data, prev, and next references.
6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are added one by one at the end of the list.
7. For the first node, both **head and tail** point to the same node. The prev of the head and the next of the tail are None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node becomes the head, its next points to the previous head, and the previous head's prev points back to the new node.

9. During **insertion at the end**, the new node is placed after the current tail. The new node's prev points to the old tail, the old tail's next points to the new node, and the new node becomes the tail.
10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the previous and next nodes, and both prev and next links are updated.
11. During **deletion by value**, the list is searched for the node containing the given value. Once found, its previous and next nodes are connected to each other, removing the selected node from the list.
12. If the node to be deleted is the **head**, the next node becomes the new head and its prev is set to None.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head. The new head's prev is set to None.
16. During **deletion at the end**, the tail node is removed and the previous node becomes the new tail. The new tail's next is set to None.
17. The **count operation** traverses the list from the head to the tail and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves forward using the next link to display all the nodes.
19. Since it is a doubly linked list, the program can also traverse the list **backwards**, starting from the tail and using the prev links.
20. The program displays the **head and tail** values to show the starting and ending nodes of the list.
21. The important relationships maintained in a doubly linked list are:

head.prev = None

tail.next = None

22. The overall structure of a doubly linked list is:

None $\leftarrow$ [10] $\rightleftarrows$ [20] $\rightleftarrows$ [30] $\rightleftarrows$ [40] $\rightarrow$ None

$\uparrow$

HEAD

$\uparrow$

TAIL

23. The arrows in both directions allow the list to be traversed **forward as well as backward**.

24. The program is **menu-driven**, allowing the user to perform creation, insertion, deletion, counting, displaying, and other operations repeatedly until the **Exit** option is selected.

Error table:

Sno	Error obtained	Rectification
1	Forgetting to update the prev pointer	Update both prev and next pointers whenever a node is inserted
2	Incorrect head.prev	Maintain head.prev = None

AIM: write a python program to implement a circular linked list which could perform the below operations by taking an input from the user

i. create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

vii. delete at the end

viii. count no of nodes

ix. display or traverse

x: display the head and tail

xi. exit

code:

cse25008.py - C:/Users/cse25008/OneDrive/Desktop/dsa lab/week4/cl.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    # i. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for _ in range(n):
            value = int(input("Enter value: "))
            self.insert_end(value)

    # ii. Insert at beginning
    def insert_beginning(self, value):
        new_node = Node(value)

        if self.head is None:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node
        else:
            new_node.next = self.head
            self.head = new_node
            self.tail.next = self.head

    # iii. Insert at end
    def insert_end(self, value):
        new_node = Node(value)
        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.tail.next = new_node
            self.tail = new_node
```

```
# iv. Insert at specific index
def insert_at_index(self, value, index):
    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_beginning(value)
        return

    if self.head is None:
        print("Index out of range")
        return

    current = self.head
    position = 0

    while position < index - 1 and current != self.tail:
        current = current.next
        position += 1

    if position != index - 1:
        print("Index out of range")
        return

    new_node = Node(value)
    new_node.next = current.next
    current.next = new_node

    if current == self.tail:
        self.tail = new_node
```

```

# v. Delete by value
def delete_by_value(self, value):
    if self.head is None:
        print("List is empty")
        return

    current = self.head
    previous = self.tail

    while True:
        if current.data == value:
            break

        previous = current
        current = current.next

    if current == self.head:
        print("Value not found")
        return

    # Only one node
    if self.head == self.tail:
        self.head = None
        self.tail = None
        return

    # Delete head
    if current == self.head:
        self.head = self.head.next
        self.tail.next = self.head
        return

    # Delete other node
    previous.next = current.next

    # Delete tail
    if current == self.tail:
        self.tail = previous

```

```

# vi. Delete at beginning
def delete_beginning(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head == self.tail:
        self.head = None
        self.tail = None
    else:
        self.head = self.head.next
        self.tail.next = self.head

# vii. Delete at end
def delete_end(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head == self.tail:
        self.head = None
        self.tail = None
        return

    current = self.head

    while current.next != self.tail:
        current = current.next

    current.next = self.head
    self.tail = current

# viii. Count nodes
def count_nodes(self):
    if self.head is None:
        return 0

    count = 0
    current = self.head

    while True:
        count += 1
        current = current.next

        if current == self.head:
            break

    return count

```

```

# ix. Display / Traverse
def display(self):
    if self.head is None:
        print("List is empty")
        return

    current = self.head

    while True:
        print(current.data, end=" -> ")
        current = current.next

        if current == self.head:
            break

    print("(Head)")

# x. Display head and tail
def display_head_tail(self):
    if self.head is None:
        print("List is empty")
        return

    print("Head:", self.head.data)
    print("Tail:", self.tail.data)
    print("Tail points to:", self.tail.next.data)

c11 = CircularLinkedList()

while True:

    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete at beginning")
    print("7. Delete at end")
    print("8. Count nodes")
    print("9. Display / Traverse")
    print("10. Display Head and Tail")
    print("11. Exit")

    choice = int(input("Enter your choice: "))

```

```
if choice == 1:
    cll.create()

elif choice == 2:
    value = int(input("Enter value: "))
    cll.insert_beginning(value)

elif choice == 3:
    value = int(input("Enter value: "))
    cll.insert_end(value)

elif choice == 4:
    value = int(input("Enter value: "))
    index = int(input("Enter index: "))
    cll.insert_at_index(value, index)

elif choice == 5:
    value = int(input("Enter value to delete: "))
    cll.delete_by_value(value)

elif choice == 6:
    cll.delete_beginning()

elif choice == 7:
    cll.delete_end()

elif choice == 8:
    print("Number of nodes:", cll.count_nodes())

elif choice == 9:
    cll.display()

elif choice == 10:
    cll.display_head_tail()

elif choice == 11:
    print("Program exited.")
    break

else:
    print("Invalid choice")
```

Output:

Creating a linked list:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter value: 1
Enter value: 2
Enter value: 3
```

Inserting at the beginning:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 2
Enter value: 4
```

Insert at the end:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 3
Enter value: 5
```

Insert at the specific index:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 4
Enter value: 3
Enter index: 2
```

Delete by value:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 5
Enter value to delete: 10
Value not found
```

Delete at the beginning:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 6
```

Delete at the end:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 7
```

Number of nodes:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 8
Number of nodes: 4
```

Display all the nodes:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 9
1 -> 3 -> 2 -> 3 -> (Head)
```

Display the head and the tail

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 10
Head: 1
Tail: 3
Tail points to: 1
```

Exit: the program:

```
----- CIRCULAR LINKED LIST -----  
1. Create linked list  
2. Insert at beginning  
3. Insert at end  
4. Insert at specific index  
5. Delete by value  
6. Delete at beginning  
7. Delete at end  
8. Count nodes  
9. Display / Traverse  
10. Display Head and Tail  
11. Exit  
Enter your choice: 11  
Program exited.
```

Explanation :

1. A **circular singly linked list** is a collection of nodes where each node contains **data** and a **link to the next node**.
2. In a circular linked list, the **last node points back to the first node (head)** instead of pointing to None.
3. The program uses two references:
 - **Head** – points to the first node.
 - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and store the data and reference to the next node.
6. During **creation of the linked list**, the user enters the number of nodes and their data. Each new node is inserted at the end of the list.
7. If the list is empty, the first node becomes both the **head and tail**, and its next points back to itself.
8. During **insertion at the beginning**, the new node is placed before the current head. The new node becomes the head, and the tail is connected to the new head.
9. During **insertion at the end**, the new node is added after the current tail. The new node becomes the tail, and its next points to the head.
10. During **insertion at a specific index**, the list is traversed until the required position is reached, and the new node is inserted between two existing nodes.

11. During **deletion by value**, the list is searched for the node containing the given value.
The node is removed by connecting its previous node directly to its next node.
12. If the node to be deleted is the **head**, the next node becomes the new head and the tail is connected to it.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is connected to the head.
14. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
15. During **deletion at the end**, the node before the tail is found and made the new tail.
16. The **count operation** traverses the list and counts all the nodes. Traversal stops when the head is reached again.
17. The **display/traverse operation** starts from the head and visits every node until it reaches the head again.
18. Since the list is circular, traversal does not use current != None. Instead, it stops when current becomes equal to head.
19. The program also displays the **head, tail, and tail.next** to show the circular connection.
20. The program is **menu-driven**, so the user can repeatedly perform different operations until the **Exit** option is selected.
21. The most important property maintained throughout the program is:

tail.next = head

22. Thus, the structure of the circular linked list is:

HEAD → Node → Node → Node → TAIL

↑ |
| _____ |

This ensures that the linked list remains **circular after every insertion and deletion operation**.

Error table:

sno	Error obtained	Rectification
1	Using temp != None as the	Stop traversal

	traversal condition	when temp == head
2	Forgetting to update tail.next when the head changes	After changing the head, update tail.next = head

WEEK-5:

5A.

Aim: write a python program to implement the stack using the arrays. And also implement stack operations such as push, pop, peek and also display the stack elements.

Code:

```
size = int(input("Enter stack size: "))
s = Stack(size)

while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        s.push(value)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Program exited.")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

Pushing an element into the stack

```
===== RESTART: C:/Users/cse25008/OneDrive/Desktop/dsa lab/week4/sll.py (3.13.7)=====
Enter stack size: 5

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 2
2 pushed into stack
```

Popping an element from the stack

```
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 2
```

Peek element

```
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Stack is empty
```

Display all the elements in the stack

```
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
3
2
```

Exiting the program

```
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program exited.
```

Explanation:

1. Class and Initialization

- A class named Stack is created.
- The `__init__()` method acts as the constructor.
- An empty list `self.stack` is created to store stack elements.
- The stack follows the **LIFO (Last In, First Out)** principle.

2. Push Operation

- The `push()` method is used to insert an element into the stack.
- The `append()` function adds the element to the end of the list.
- The newly inserted element becomes the **top element**.
- A message is displayed after the element is inserted.

3. Pop Operation

- The `pop()` method is used to remove the top element.
- The program first checks whether the stack is empty.
- If the stack is empty, **Stack Underflow** is displayed.

- Otherwise, the pop() function removes the last element.
- The removed element is displayed.

4. Peek Operation

- The peek() method is used to view the top element.
- The program checks whether the stack is empty.
- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.stack[-1] is used to access the top element.
- The top element is displayed without removing it.

5. Display Operation

- The display() method is used to display all stack elements.
- The program first checks whether the stack is empty.
- If empty, **Stack is empty** is displayed.
- Otherwise, a for loop is used to traverse the stack.
- The elements are displayed from **top to bottom**.

6. Count Operation

- The count() method is used to find the number of elements in the stack.
- The len() function returns the number of elements.
- The count is displayed to the user.

Error table:

sno	Error obtained	Error rectification
1	Stack underflow	Check whether the stack is empty before popping
2	Name error	Define the class before creating the object

5b.

Aim: write a python program to implement the stack using the linked list. And also implement stack operations such as push, pop, peek, count and also display the stack elements.

Code:

```
*2.py - C:/Users/cse25008/OneDrive/Desktop/dsa lab/week5/2.py (3.13.7)*
File Edit Format Run Options Window Help
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None
        self.count = 0

    # Push operation
    def push(self, value):
        new_node = Node(value)

        new_node.next = self.top
        self.top = new_node
        self.count += 1

        print(value, "pushed into stack")

    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
            return

        value = self.top.data
        self.top = self.top.next
        self.count -= 1

        print("Popped element:", value)
```

```
# Peek operation
def peek(self):
    if self.top is None:
        print("Stack is empty")
    else:
        print("Top element:", self.top.data)

# Count operation
def count_nodes(self):
    print("Number of elements:", self.count)

# Display operation
def display(self):
    if self.top is None:
        print("Stack is empty")
        return

    current = self.top

    print("Stack elements:")

    while current:
        print(current.data)
        current = current.next

stack = Stack()
```

```

while True:

    print("\n----- STACK USING LINKED LIST -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Count")
    print("5. Display")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        stack.push(value)

    elif choice == 2:
        stack.pop()

    elif choice == 3:
        stack.peek()

    elif choice == 4:
        stack.count_nodes()

    elif choice == 5:
        stack.display()

    elif choice == 6:
        print("Program exited.")
        break

    else:
        print("Invalid choice")

```

Output:

Pushing an element into the stack

----- STACK USING LINKED LIST -----

1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit

Enter your Choice: 1

Enter value: 2

2 pushed into stack

Popping an element from the stack

----- STACK USING LINKED LIST -----

1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit

Enter your choice: 2

Popped element: 5

Peek element

----- STACK USING LINKED LIST -----

1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit

Enter your choice: 3

Top element: 3

Display all the elements in the stack

```
----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 5
Stack elements:
3
2
```

count

```
----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 4
Number of elements: 2
```

Exiting the program

```
----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 6
Program exited.
```

Explanation:

1. Node Class

- A class named Node is created to represent each element of the linked list.
- The `__init__()` method initializes a node.
- `self.data` stores the value of the node.
- `self.next` stores the reference to the next node.

- Initially, self.next is set to None.

2. Stack Class

- A class named Stack is created to implement the stack operations.
- The `__init__()` method initializes the stack.
- `self.top = None` indicates that the stack is initially empty.
- The top variable stores the reference to the top node of the stack.

3. Push Operation

- The `push()` method is used to insert an element into the stack.
- A new node is created using `Node(data)`.
- The new node is connected to the current top node using:

```
new_node.next = self.top
```

- The new node is then made the top node using:

```
self.top = new_node
```

- The inserted element is displayed.
- The operation follows the **LIFO (Last In, First Out)** principle.

4. Pop Operation

- The `pop()` method is used to remove the top element.
- The program first checks whether `self.top` is None.
- If `self.top` is None, the stack is empty and **Stack Underflow** is displayed.
- Otherwise, the data of the top node is stored in data.
- The top is moved to the next node using:

```
self.top = self.top.next
```

- The removed element is displayed.

5. Peek Operation

- The `peek()` method is used to view the top element without removing it.
- The program checks whether the stack is empty.

- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.top.data is used to display the top element.
- The stack remains unchanged after the peek operation.

6. Display Operation

- The display() method is used to display all the elements of the stack.
- The program first checks whether the stack is empty.
- A temporary variable current is initialized with self.top.
- A while loop is used to traverse the linked list.
- current.data displays the value of each node.
- current = current.next moves to the next node.
- The elements are displayed from **top to bottom**.

7. Count Operation

- The count() method is used to find the number of elements in the stack.
- A variable count is initialized to 0.
- A temporary variable current starts from self.top.
- The linked list is traversed using a while loop.
- The count value is increased for every node.
- The total number of elements is displayed.

Error table:

sno	Error obtained	Error rectification
1	Incorrect push	Set new_node.next = self.top before updating self.top
2	Infinte loop	Use break when choice == 6

